

## Code-Layout, Readability and Reusability

|                      |                                                 |
|----------------------|-------------------------------------------------|
| <b>Date</b>          | 4 July 2026                                     |
| <b>Team ID</b>       | XXXXXX                                          |
| <b>Project Name</b>  | OptiCrop – Smart Agricultural Production Engine |
| <b>Maximum Marks</b> | 5 Marks                                         |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                       |
|------|---------------------------|-----------------------------------------------|-------------------------------|-------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing                               | Yes                           | Code follows consistent indentation.                                          |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Project contains templates, static, dataset, model, and app.py.               |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Variables such as temperature, humidity, rainfall, prediction are meaningful. |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions clearly indicate their purpose (e.g., prediction and routing).      |
| 5    | Code Comments             | Inline and block comments explain logic       | Partial                       | Important sections are commented, but more comments can be added.             |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Frontend, backend, dataset, and model are separated into different modules.   |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | No unnecessary duplicate code.                                                |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Partial                       | Basic validation is implemented; advanced exception handling can be improved. |

### Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused             | Reusability Level (High / Medium / Low) |
|------|-------------------------|-----------------------|--------------------------|-----------------------------------------|
| 1    | app.py                  | Python (Flask)        | Backend request handling | High                                    |

|   |                             |              |                            |        |
|---|-----------------------------|--------------|----------------------------|--------|
| 2 | index.html                  | HTML         | User Interface             | High   |
| 3 | style.css                   | CSS          | Web page styling           | High   |
| 4 | model.pkl                   | Scikit-learn | Crop prediction            | High   |
| 5 | Crop Recommendation Dataset | CSV          | Model training and testing | Medium |

Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                               |
|--------------------------|--------------|------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-organized project structure.                                      |
| Readability              | 5            | Code is easy to understand and maintain.                               |
| Reusability              | 4            | Components can be reused in future agricultural projects.              |
| Documentation / Comments | 4            | Basic comments are available; additional comments can improve clarity. |
| Overall Score            | 4.5/5        | The project follows good coding practices and is maintainable.         |